



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

Modelo de Classes - CSU04

Designer de Sistemas

Equipe 03 - Jogo Desconecta

Ana Lúcia de Souza - 5084

Florestal

2025

Sumário

CSU04:Solucionar Desafio.....	3
Back-end.....	3
Front-end.....	6

Back-end

[illegible]

```

classDiagram
    class Usuario
    class ControladorDesafio {
        - acessoBDDesafio : AcessoBDDesafio
        - solucionarDesafio : Map<String, SolucionarDesafio>
        + concluirDesafio(idDesafio : int) : boolean
        + salvarPontuacaoDesafio(apelidoAluno : String, nomeLinha : String, tempo : int, numErros : int) : int
        + verificar(tipoDesafio : EnumTiposDesafios, id : int, tentativa : String) : boolean
    }
    class ControladorConecta {
        <<Control>>
        - acessoBDConecta : AcessoBDConecta
        + verificarAgrupamento(idDesafio : int, tentativa : String) : String
        + atualizarRespostaCerta(idConecta : int, novaResposta : String, resposta : String) : boolean
    }
    class ControladorCacaPalavras {
        <<Control>>
        - acessoBDCacaPalavras : int
        + verificarAgrupamento(idDesafio : int, tentativa : String) : String
    }
    class AcessoBDConecta {
        <<Entity>>
        - idConecta : int
        - respostaCerta1 : String
        - respostaCerta2 : String
        - respostaCerta3 : String
        - respostaCerta4 : String
    }
    class AcessoBDCacaPalavras {
        <<Entity>>
        - idCacaPalavras : int
        - respostaCerta : List<String>
    }
    class SolucionarDesafio {
        <<interface>>
        + verificarAgrupamento(idDesafio : int, tentativa : String) : String
    }
    Usuario ..> ControladorDesafio
    ControladorDesafio o--> ControladorConecta
    ControladorDesafio o--> ControladorCacaPalavras
    ControladorDesafio o--> SolucionarDesafio
    ControladorConecta ..|> SolucionarDesafio
    ControladorCacaPalavras ..|> SolucionarDesafio
    ControladorConecta o--> AcessoBDConecta
    ControladorCacaPalavras o--> AcessoBDCacaPalavras

```

Figura 2. Diagrama de Classes para CSU04.

Como pode ser observado na figura 2, para este caso de uso, faremos o uso da interface “**SolucionarDesafio**”. Essa interface armazenará os métodos obrigatórios que devem ser implementados por todos os jogos, neste caso, o método “**verificarAgrupamento()**”. O método “**atualizarRespostaCerta()**” também pode ser um desses métodos, caso os desenvolvedores julguem necessário a sua implementação em Caça-palavras. Como a utilidade desse método é secundária, ou seja, ele não compõe a lógica do jogo, apenas auxilia na mudança das respostas internamente, sua implementação pode ser considerada facultativa.

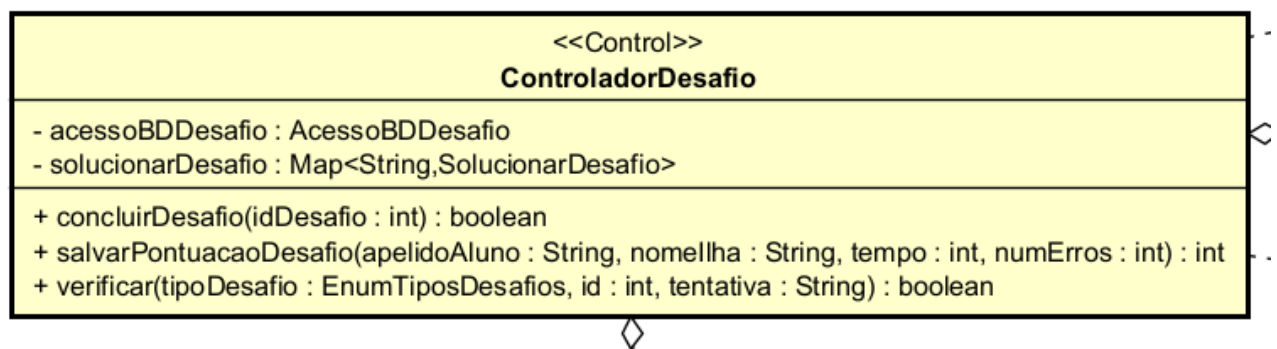


Figura 3. Classe ControladorDesafio.

Além disso, observe a classe “**ControladorDesafio**”, em destaque na figura 3. Nela, há o novo atributo **solucionarDesafio** que deve ser implementado como um Map. Assim, a declaração **Map<String, SolucionarDesafio>** define um Mapa, que é como um dicionário ou uma lista de contatos, armazenando pares de Chave-Valor. Nesse caso, a String define a chave e deve ser composta pelo tipo do desafio presente no Enum, ou seja “**JogoConecta**” e “**JogoPalavras**”. Já SolucionarDesafio será o valor e se constitui como o objeto real que está sendo armazenado e que queremos recuperar usando a chave. Em outras palavras, SolucionarDesafio é interface. O valor armazenado será um objeto de uma classe que implementa essa interface (como um objeto ControladorConecta ou um objeto ControladorCacaPalavras).

Talvez você esteja se perguntando o motivo de usarmos uma interface e a resposta para essa pergunta é que aqui estamos usando uma adaptação do strategy, um padrão de projeto comportamental. Caso deseje, você pode estudá-lo e entendê-lo melhor acessando este [link](#), mas nossa implementação independe do conhecimento do padrão em si. A adaptação se deve ao uso do map (falaremos dele novamente em breve) e a não termos uma classe cliente propriamente dita, ela está implícita nas “magias” do spring boot. No nosso caso, Contexto (quem usa a estratégia) é a classe **ControladorDesafio**, a

Estratégia é a interface **SolucionarDesafio** e as Estratégias Concretas são as implementações, ou seja, **ControladorConecta** e **ControladorCacaPalavras**. Saber qual classe representa qual dentro do strategy não é, de forma alguma, absolutamente necessário aos desenvolvedores neste momento, apenas tenham em mente que o uso de um padrão de projeto é importante e que seu uso poderia nos facilitar em muito caso quiséssemos, no futuro, adicionar mais jogos ao nosso projeto.

Mas por que usar um Map? Talvez outras implementações também funcionassem aqui, mas ao usar um Map estamos usando a Injeção de Dependência do Spring - um mecanismo do framework que transfere a responsabilidade de criar e fornecer as dependências de uma classe para um componente externo, reduzindo o acoplamento e aumentando a flexibilidade e a testabilidade do código. Em vez de uma classe instanciar seus próprios objetos de dependência, eles são "injetados" nela, permitindo que a classe trabalhe com abstrações em vez de implementações concretas.

Dessa forma, a classe ControladorDesafio não vai criar o Map de estratégias. Ela **não** vai dar `new HashMap<>()` e preenchê-lo manualmente. Em vez disso, ela vai "pedir" ao Spring por esse mapa, provavelmente com um `@Autowired`. O spring vai ver o pedido, criar o Map, encontrar todos os seus beans (ControladorConecta, ControladorCacaPalavras) que implementam SolucionarDesafio, colocá-los dentro do Map e injetar o mapa pronto dentro da nossa classe ControladorDesafio. O controle sobre a criação e fornecimento do Map então foi invertido, ou seja, não é a classe ControladorDesafio que controla, é o Spring.

Como vocês já devem estar cientes a essa altura do campeonato, um bean é um objeto que o Spring gerencia e para criá-lo usamos a anotação `@Service`. Aqui, será preciso que o bean criado não tenha o mesmo nome da classe como é feito por padrão. Precisaremos mudar seu nome para o mesmo do Enum, ou seja, JogoConecta ou JogoPalavras. Então para isso teremos que fazer `@Service("JogoConecta")` como no exemplo:

```
Java
@Service("JogoConecta")
public class ControladorCacaPalavras implements SolucionarDesafio{
}
```

Desse modo, ao usarmos o `Map<String, SolucionarDesafio>` é como se pedíssemos ao spring para procurar por todos os beans que implementem SolucionarDesafio. No nosso caso, ele encontrará dois: o objeto ControladorConecta, que

tem o rótulo "JogoConecta" - porque foi anotado com `@Service("JogoConecta")` e o objeto `ControladorCacaPalavras`, que tem o rótulo "JogoPalavras" - porque foi anotado com `@Service("JogoPalavras")`. Assim, a chave será constituída pelo nome do bean e o map será construído automaticamente pelo spring. Por fim, ele injeta (entrega) esse mapa já preenchido na variável `solucionarDesafio`.

Agora, ainda na classe `ControladorDesafio`, vamos falar do método **`verificar()`**. A primeira coisa que é preciso destacar a respeito desse método é que ele recebe um Enum no parâmetro e nosso map tem chave do tipo string, então, para que isso funcione é preciso comparar se o nome do Enum, ou seja, `JogoConecta` ou `JogoPalavras` é igual a string que corresponde a chave no mapa. O método `verificar()` tem como objetivo chamar um dos métodos `verificarAgrupamento()`, ou seja, ou chama o `verificarAgrupamento` da classe `ControladorConecta` ou o da classe `ControladorCacaPalavras`. Mas como nosso método `verificar()` vai saber qual dos dois chamar? É aqui que tudo se conecta! O tipo recebido como parâmetro será usado como a chave para selecionar o objeto do valor `SolucionarDesafio` correspondente. Explicando melhor: vamos supor que estamos no jogo `Conecta`. Ao clicar no botão `verificar` depois de selecionar um agrupamento, o método `verificar()` deve receber do front o id do desafio, a string formada pela tentativa de resposta do aluno e também o `tipoDesafio`, que é o Enum. O `tipoDesafio` nesse caso é `JogoConecta`. Deve-se verificar no map qual dos dois objetos lá presente possui a chave "JogoConecta". O objeto correspondente, ou seja, o objeto do tipo `ControladorConecta` será retornado pelo método `get` do map e, por fim, o método `verificarAgrupamento()` usado será o de `ControladorConecta`.

Front-end

Para a modelagem do front-end, foi utilizado o diagrama de máquina de estados conforme apresentado na figura 4. Nesse diagrama, **os estados representam as telas ou popUps e as transições representam os botões**.

Você pode acessar o diagrama da figura 4 através de seu e-mail institucional clicando [aqui](#).

Como este caso de uso não implementa nada novo, apenas une as partes do sistema, não há nada para acrescentar no diagrama que já não tenha sido realizado. Assim, o diagrama abaixo foi montado apenas unindo partes do nosso sistema construído até então. Observe que optei por deixar a expansão menu separado do restante do programa, mas o menu está indicado em ambos os jogos.

